

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Learning Outcomes

Students will be able to:

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3. Background Theory

A Convolutional Neural Network (CNN) consists of the following major components:

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is given by:

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where:

- X represents the input image,
- K represents the kernel or filter,
- Y represents the resulting feature map.

The output feature-map size is calculated using:

$$O = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

where:

- N is the input size,
- F is the filter size,
- P is the padding,
- S is the stride.

3.1 Common Convolution Kernels

A kernel (or filter) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$K = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose: Preserves the original image and is used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects vertical edges by measuring intensity changes along the horizontal direction.

$$K = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications: Object detection, lane detection, and face recognition.

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects horizontal edges by measuring intensity changes along the vertical direction.

$$K = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications: Text detection, boundary extraction, and medical image analysis.

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$K = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Application: Medical imaging and edge detection.

5. Sharpening Kernel

The sharpening kernel enhances fine details by emphasizing high-frequency components.

$$K = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications: Image enhancement, Optical Character Recognition (OCR), and face recognition.

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$K = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications: Noise removal, image smoothing, and image preprocessing.

7. Gaussian Blur Kernel

The Gaussian Blur kernel smooths the image while preserving the overall structure better than a simple average filter.

$$K = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications: Noise reduction, computer vision preprocessing, and feature extraction.

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$K = \begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications: Texture analysis and artistic image effects.

9. Outline Detection Kernel

The Outline Detection kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$K = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications: Image segmentation, boundary detection, and shape extraction.

10. Motion Blur Kernel

The Motion Blur kernel simulates motion blur along the diagonal direction.

$$K = \frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications: Motion simulation, image restoration, and video processing.

4. Numerical Example 1: Convolution

Consider the following input image:

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

The kernel is:

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

The convolution operation is performed by sliding the kernel over the input image.

First position:

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position:

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position:

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position:

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Therefore, the resulting feature map is:

$$\boxed{\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}}$$

5. Numerical Example 2: Max Pooling

Consider the following feature map:

$$X = \begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter, the maximum value from each window is selected.

Window 1:

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2:

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3:

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4:

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Therefore, the output after max pooling is:

$$\boxed{\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}}$$

6. Numerical Example 3: Parameter Calculation

Consider an input image of size:

$$32 \times 32 \times 3$$

where the input has three channels corresponding to an RGB image.
The convolution layer contains:

- Number of filters = 16
- Kernel size = 3×3

The number of trainable parameters is calculated as:

$$(3 \times 3 \times 3 + 1) \times 16$$

$$= (27 + 1) \times 16$$

$$= 28 \times 16$$

$$\boxed{448}$$

Therefore, the total number of trainable parameters in the convolution layer is:

$$448$$

7. Dataset

The CIFAR-10 dataset is used for this experiment. It consists of color images belonging to ten different classes.

Table 1: CIFAR-10 Dataset Details

Parameter	Value
Dataset	CIFAR-10
Training Images	50,000
Testing Images	10,000
Number of Classes	10
Image Size	$32 \times 32 \times 3$
Image Type	RGB

The ten classes in the CIFAR-10 dataset are:

Class ID	Class Name
0	Airplane
1	Automobile
2	Bird
3	Cat
4	Deer
5	Dog
6	Frog
7	Horse
8	Ship
9	Truck

8. Experimental Procedure

Task 1: Dataset Exploration

The CIFAR-10 dataset was explored to understand its dimensions, class distribution, and image characteristics. Sample images from the dataset were displayed and the number of samples belonging to each class was analyzed.

The dataset contains 50,000 training images belonging to 10 classes. Each image has a spatial dimension of 32×32 pixels and three RGB channels.

Figure 1 shows representative sample images from the CIFAR-10 dataset.



Figure 1: Sample Images from the CIFAR-10 Dataset

The class distribution was also visualized to verify the distribution of samples among the ten classes.

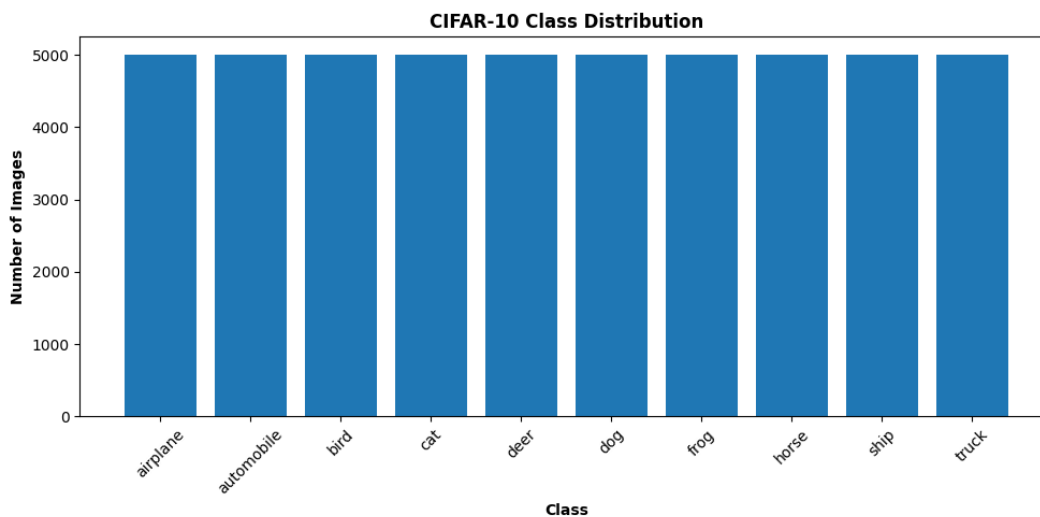


Figure 2: Class Distribution of the CIFAR-10 Dataset

Task 2: Convolution

Convolution was performed using kernels of different sizes to study their effect on the resulting feature maps. The kernel sizes considered were 3×3 , 5×5 , and 7×7 .

The convolution operation applies a kernel over the input image and computes weighted sums at each spatial location. The resulting feature map represents the features detected by the corresponding kernel.

The output feature-map size was calculated using:

$$O = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

where N is the input size, F is the kernel size, P is the padding, and S is the stride. The feature maps obtained using different convolution kernels were compared visually.

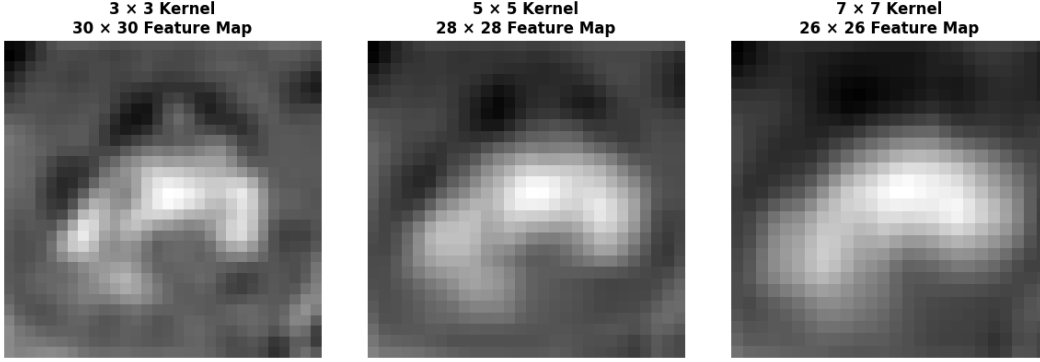


Figure 3: Comparison of Feature Maps Using Different Convolution Kernels

Inference: Different kernel sizes capture spatial features at different scales. Smaller kernels focus on local patterns, while larger kernels cover a wider spatial region and can capture more extensive features.

Task 3: Stride and Padding

The effects of stride and padding on the spatial dimensions of convolution output were studied.

The following configurations were compared:

- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

The output dimensions were calculated using the convolution output-size formula.

Inference: Increasing the stride reduces the spatial dimensions of the feature map. Same padding preserves spatial dimensions under appropriate conditions, whereas valid padding does not add padding and therefore generally produces a smaller output.

Task 4: Feature Map Visualization

Feature maps generated after the first convolution layer were visualized to understand the features learned by the convolution filters.

The first convolution layer applies multiple filters to the input image. Each filter responds to different visual characteristics present in the image, resulting in different feature maps.

At least eight feature maps were selected and displayed for visualization.

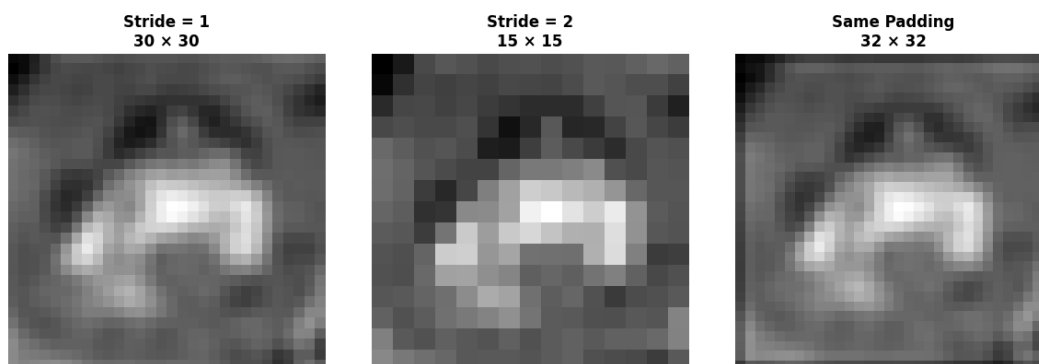


Figure 4: Effect of Stride and Padding on Convolution Output

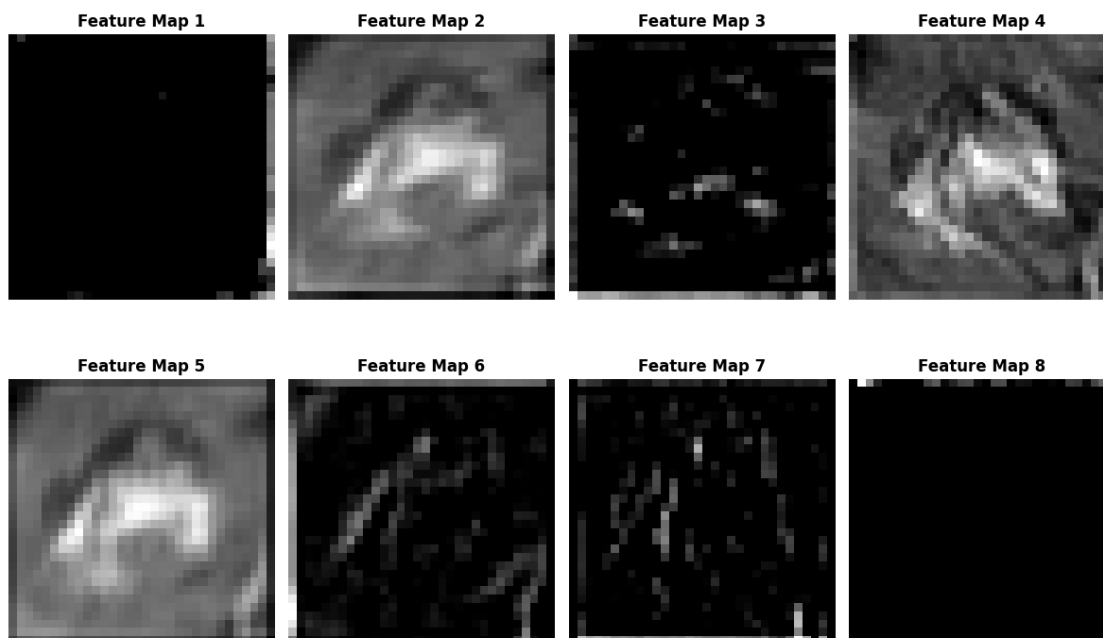


Figure 5: Feature Maps Generated by the First Convolution Layer

Inference: The different feature maps show that individual convolution filters respond differently to patterns present in the input image. Some filters emphasize edges and local structures, while others capture different visual patterns.

Task 5: Pooling

Pooling is used to reduce the spatial dimensions of feature maps while retaining important information. In this experiment, Max Pooling and Average Pooling were compared using a 2×2 pooling window.

Max Pooling selects the maximum value from each pooling region, whereas Average Pooling calculates the average value of the elements within the region.

The output dimensions of both pooling methods were compared.

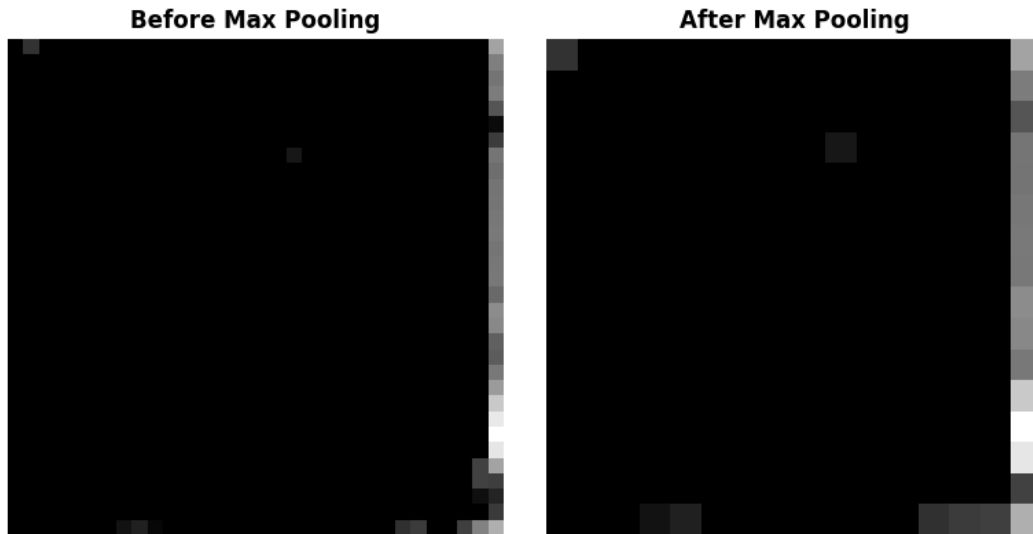


Figure 6: Comparison of Max Pooling and Average Pooling

The classification performance of the two pooling methods was also evaluated.

Table 2: Max Pooling and Average Pooling Accuracy

Pooling Method	Test Accuracy
Max Pooling	67.03%
Average Pooling	67.59%

Inference: Both pooling methods reduce the spatial dimensions of the feature maps. In the experiment, Average Pooling achieved slightly higher test accuracy than Max Pooling, with accuracies of 67.59% and 67.03%, respectively.

Task 6: CNN Construction and Training

A Convolutional Neural Network was constructed for multi-class image classification using the CIFAR-10 dataset. The CNN architecture consists of convolutional layers, ReLU activation func-

tions, max pooling layers, a flattening layer, a fully connected dense layer, and a Softmax output layer.

The architecture of the implemented CNN is:

Input $\rightarrow$ Conv2D $\rightarrow$ ReLU $\rightarrow$ MaxPooling $\rightarrow$ Conv2D $\rightarrow$ ReLU $\rightarrow$ MaxPooling $\rightarrow$ Flatten $\rightarrow$ Dense $\rightarrow$ Softmax

The model was compiled using the Adam optimizer and trained for 20 epochs with a batch size of 32.

Table 3: CNN Training Configuration

Parameter	Value
Input Size	$32 \times 32 \times 3$
Optimizer	Adam
Epochs	20
Batch Size	32
Output Classes	10
Total Parameters	545,098

The model was trained using the training dataset, with a validation subset used to monitor the model's performance during training.

The training and validation accuracy were recorded for each epoch.

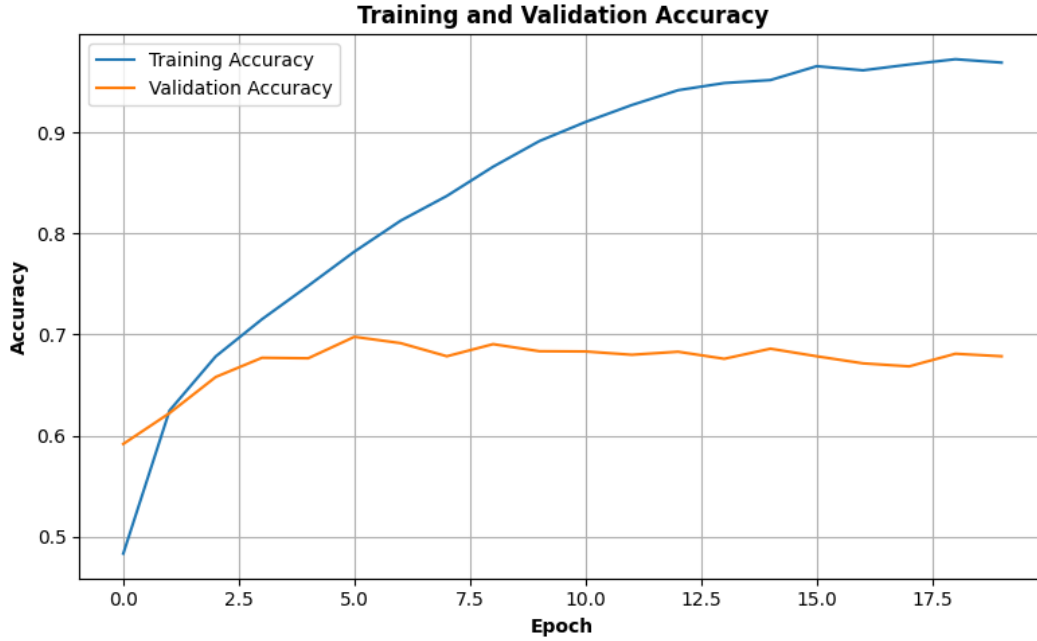


Figure 7: Training and Validation Accuracy

The training and validation loss were also recorded throughout the training process.

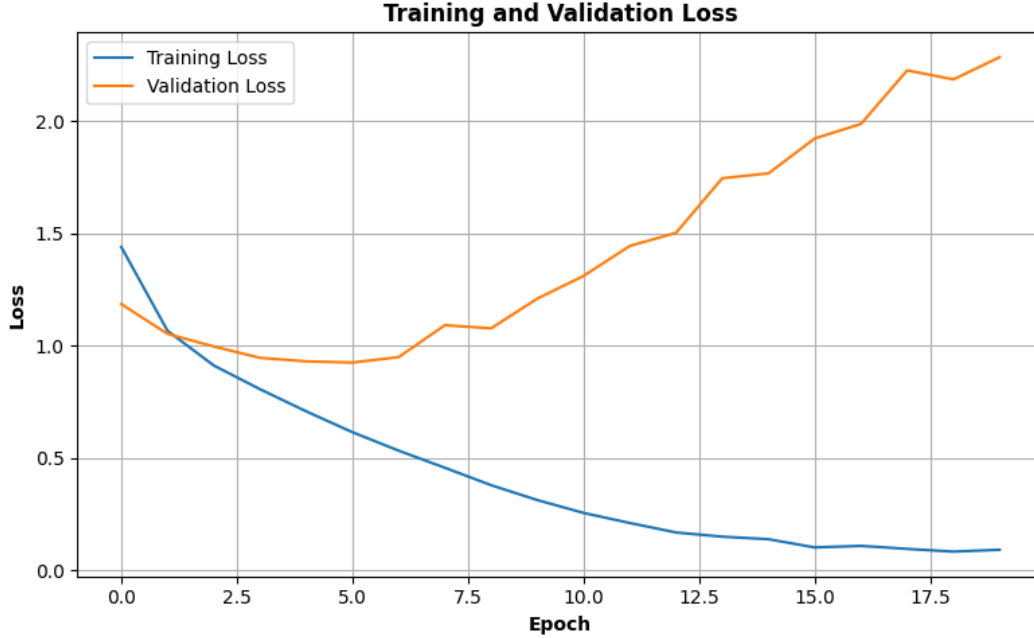


Figure 8: Training and Validation Loss

Inference: The training accuracy increased substantially during training, reaching approximately 97% by the final epoch. However, the validation accuracy remained considerably lower, while the validation loss increased during the later epochs, indicating overfitting.

Task 7: Model Evaluation

The trained CNN was evaluated on the testing dataset to measure its classification performance. The evaluation metrics used were accuracy, precision, recall, and F1-score. A confusion matrix and classification report were also generated to analyze the classification performance across the ten CIFAR-10 classes.

The model achieved the following results on the testing dataset:

Table 4: CNN Test Performance

Metric	Value
Testing Accuracy	67.03%
Precision	67.18%
Recall	67.03%
F1-score	66.97%

The confusion matrix was generated to identify the correct and incorrect predictions made for each of the ten classes.

Inference: The confusion matrix shows the distribution of correct and incorrect predictions across the ten CIFAR-10 classes. The diagonal elements represent correctly classified images, while the off-diagonal elements represent misclassified images.

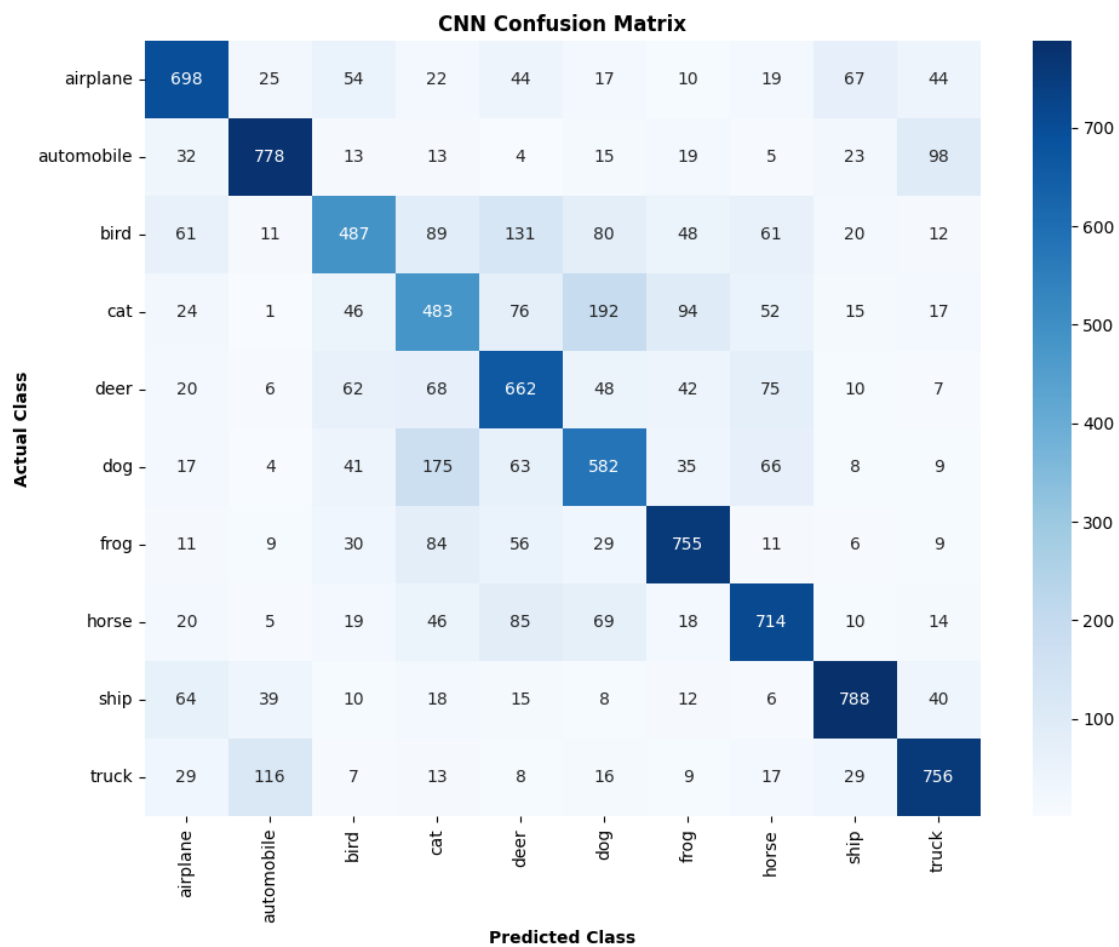


Figure 9: Confusion Matrix of the CNN Model

The classification report provides class-wise precision, recall, F1-score, and support values for the ten classes. It provides a detailed view of the model's performance for individual categories.

Source Code

The complete source code for this experiment, including dataset preprocessing, convolution operations, stride and padding experiments, feature map visualization, pooling comparison, CNN construction, training, evaluation, and additional exercises, is available in the GitHub repository.

<https://github.com/KPR-24110081/Deep-Learning-Lab/tree/main/Ex-3>

The repository contains the complete Jupyter Notebook, generated figures, documentation, and supporting files used for this experiment.

9. Mandatory Plots

The following plots were generated to analyze the CNN training process, dataset characteristics, learned feature maps, and classification performance.

9.1 Sample Images



Figure 10: Sample Images from the CIFAR-10 Dataset

Inference: The sample images demonstrate the visual characteristics of the ten CIFAR-10 classes. The images have a size of 32×32 pixels with three RGB channels.

9.2 Class Distribution

Inference: The class distribution shows that the CIFAR-10 dataset contains an equal number of samples for each class. Therefore, the dataset is balanced across the ten categories.

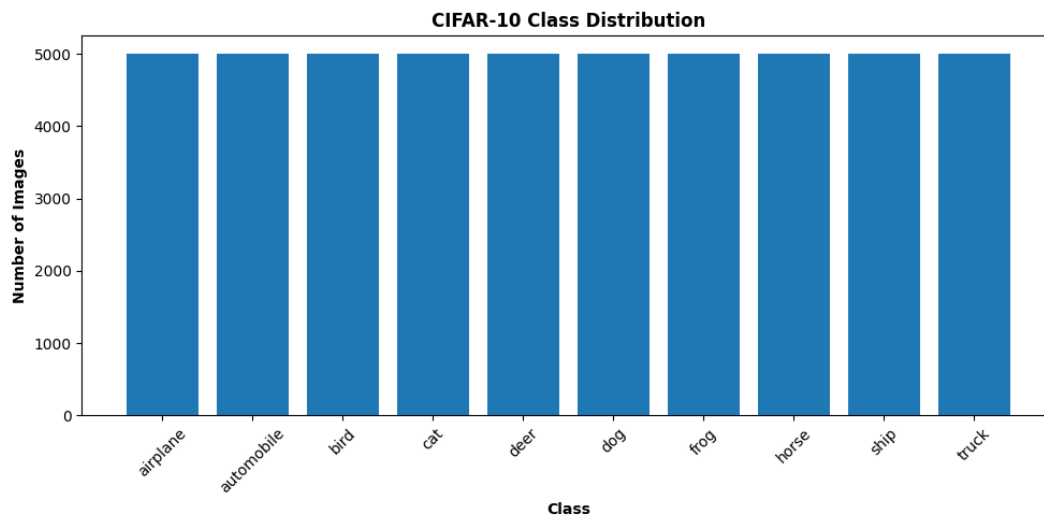


Figure 11: CIFAR-10 Class Distribution

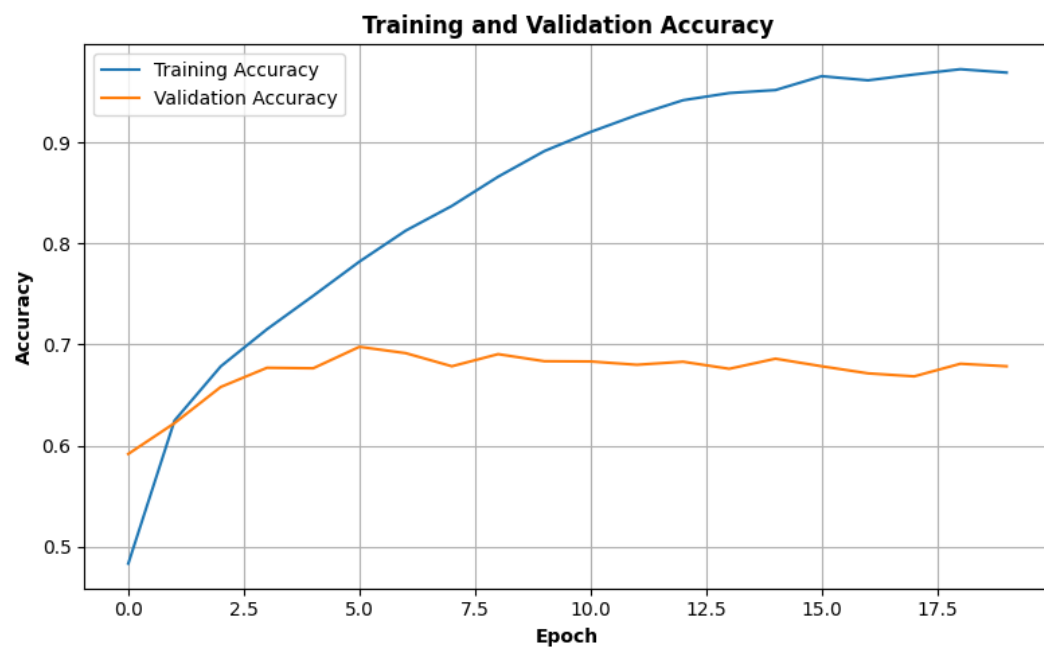


Figure 12: Training and Validation Accuracy

9.3 Training Accuracy

The training accuracy curve is shown together with the validation accuracy curve in Figure 12.

Inference: The training accuracy increased substantially throughout the training process and reached approximately 97% by the final epoch. This indicates that the CNN learned the training samples effectively.

9.4 Validation Accuracy

The validation accuracy is shown in the same training-history plot in Figure 13.

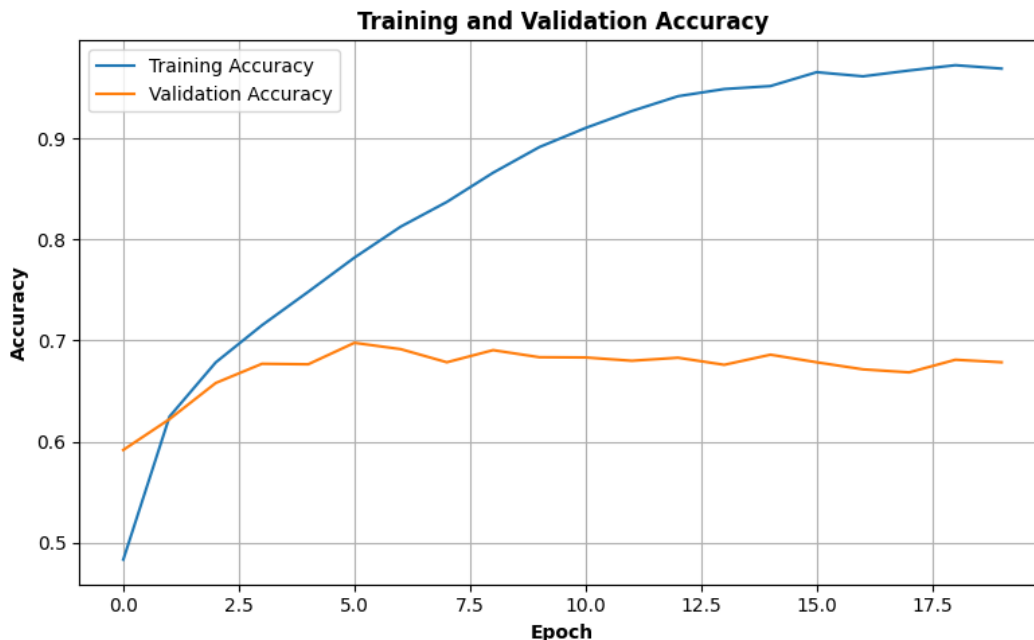


Figure 13: Validation Accuracy During Training

Inference: The validation accuracy improved during the earlier stages of training but remained substantially below the training accuracy. The increasing gap between the two curves indicates that the model experienced overfitting.

9.5 Training Loss

The training loss curve is shown together with the validation loss curve in Figure 14.

Inference: The training loss decreased considerably during training and reached approximately 0.09 by the final epoch. This indicates that the model progressively reduced its error on the training data.

9.6 Validation Loss

The validation loss is shown in the same training-history plot in Figure 15.

Inference: The validation loss initially decreased but increased during the later epochs, reaching approximately 2.28 at the final epoch. The divergence between training and validation loss indicates overfitting.



Figure 14: Training and Validation Loss



Figure 15: Validation Loss During Training

9.7 Feature Maps

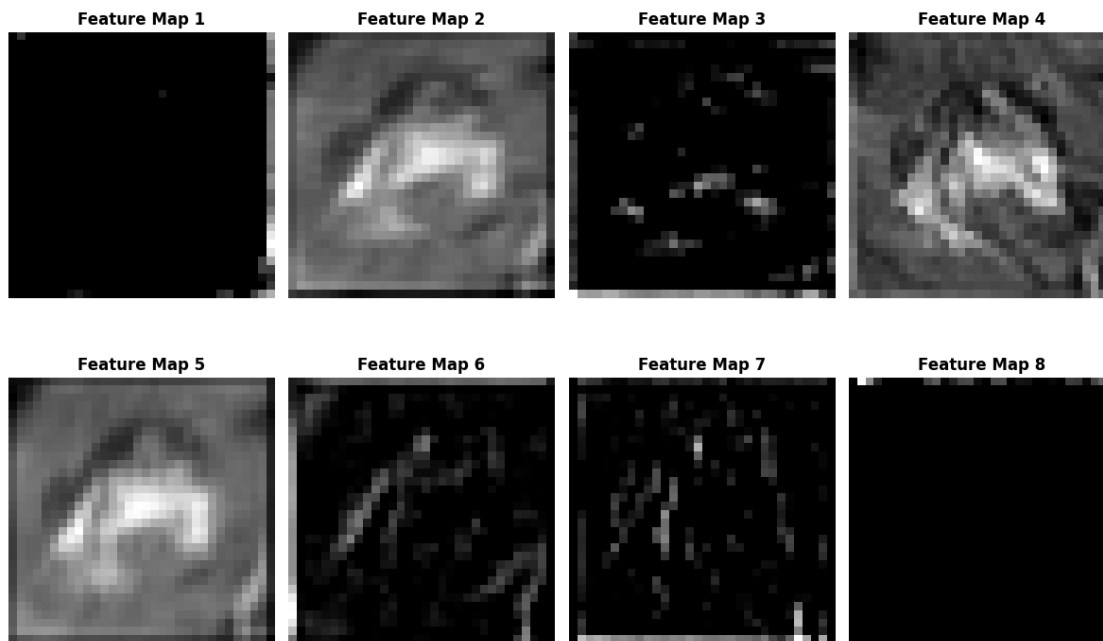


Figure 16: Feature Maps Generated by the Convolution Layer

Inference: The feature maps demonstrate that different convolution filters respond to different patterns in the input image. The filters extract visual information that is subsequently used by deeper layers for classification.

9.8 Confusion Matrix

Inference: The diagonal elements of the confusion matrix represent correctly classified images, while the off-diagonal elements represent misclassifications. The matrix shows that classification performance varies across the different CIFAR-10 classes.

10. Additional Exercises

10.1 Convolution Output Size

For a 64×64 input image with a 5×5 kernel, stride 2, and padding 2, the output size is calculated using:

$$O = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

Substituting the given values:

$$O = \left\lfloor \frac{64 - 5 + 2(2)}{2} \right\rfloor + 1$$

$$O = \left\lfloor \frac{63}{2} \right\rfloor + 1$$

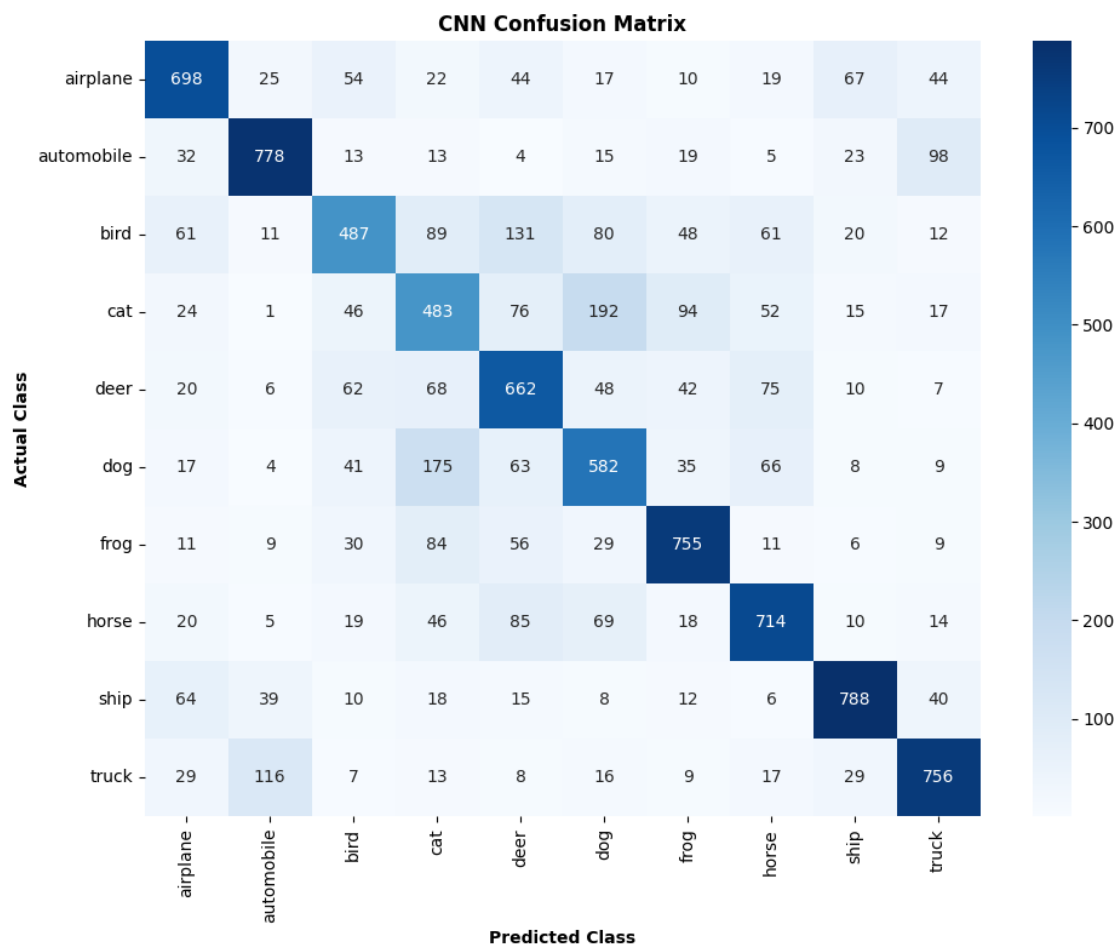


Figure 17: Confusion Matrix of the CNN

$$O = 31 + 1$$

$$O = 32$$

Therefore, the output feature-map size is:

$$32 \times 32$$

10.2 Convolution Layer Parameter Calculation

Consider a convolution layer with 64 filters of size 3×3 and an RGB input.

Since an RGB image has three input channels, the number of parameters is:

$$(3 \times 3 \times 3 + 1) \times 64$$

$$= (27 + 1) \times 64$$

$$= 28 \times 64$$

$$1792$$

Therefore, the convolution layer contains 1,792 trainable parameters, including the bias parameter for each filter.

10.3 Comparison of ReLU and Sigmoid

The ReLU activation function is defined as:

$$f(x) = \max(0, x)$$

The Sigmoid activation function is defined as:

$$f(x) = \frac{1}{1 + e^{-x}}$$

ReLU produces zero for negative inputs and retains positive values, whereas Sigmoid maps the input to a value between 0 and 1.

Table 5: Comparison of ReLU and Sigmoid

Property	ReLU	Sigmoid
Output Range	$[0, \infty)$	$(0, 1)$
Negative Input	0	Positive value
Typical Use	Hidden Layers	Binary Output
Computation	Simple	More computationally expensive

For the CNN used in this experiment, ReLU was used as the activation function in the convolutional and dense hidden layers.

10.4 Max Pooling versus Average Pooling

Max Pooling and Average Pooling were compared experimentally using the CNN architecture.

Table 6: Pooling Method Comparison

Pooling Method	Test Accuracy
Max Pooling	67.03%
Average Pooling	67.59%

Average Pooling achieved a slightly higher test accuracy than Max Pooling in this experiment. Both methods reduce the spatial dimensions of the feature maps, but they summarize the activation values differently.

10.5 Effect of Increasing Convolution Filters

The effect of increasing the number of convolution filters from 16 to 64 was analyzed while keeping the other major training settings constant.

Table 7: Effect of Number of Convolution Filters

Configuration	Test Accuracy	Training Time (s)
16 Filters	63.48%	158.04
64 Filters	67.25%	340.12

Increasing the number of filters from 16 to 64 improved the test accuracy from 63.48% to 67.25%. However, the training time increased from 158.04 seconds to 340.12 seconds because the larger number of filters increases the computational requirements of the convolution layers.

11. Results

The performance of the CNN model was evaluated using the training and testing datasets. The obtained performance metrics are summarized below.

Table 8: CNN Performance Results

Metric	Value
Training Accuracy	96.91%
Testing Accuracy	67.03%
Precision	67.18%
Recall	67.03%
F1-score	66.97%
Number of Parameters	545,098

The CNN achieved a training accuracy of 96.91% and a testing accuracy of 67.03%. The precision, recall, and F1-score were 67.18%, 67.03%, and 66.97%, respectively. The constructed CNN contains 545,098 trainable parameters.

12. Discussion

1. Why is convolution preferred over fully connected layers for images?

Convolution is preferred for image data because it exploits the spatial structure of images and focuses on local regions. The same convolutional filter is shared across different locations, which significantly reduces the number of trainable parameters compared with fully connected layers while allowing the network to learn important spatial features.

2. How does stride affect the feature map size?

Stride determines the number of pixels by which the convolution filter moves across the input. Increasing the stride reduces the spatial dimensions of the resulting feature map. Therefore, a larger stride produces a smaller feature map and reduces the number of convolution operations.

3. What is the role of padding?

Padding adds additional pixels around the input before applying convolution. It can help preserve spatial information and control the size of the resulting feature map. Same padding can maintain the spatial dimensions under appropriate conditions, whereas valid padding does not add extra pixels.

4. Why is pooling used?

Pooling is used to reduce the spatial dimensions of feature maps while retaining important information. It reduces computational requirements and can make the learned representation less sensitive to small spatial changes in the input.

5. How do feature maps represent image characteristics?

Feature maps represent the responses produced by convolution filters when they are applied to an input image. Different filters learn different visual patterns such as edges, textures, and other local structures. The collection of feature maps therefore provides a representation of the characteristics detected by the convolutional layers.

6. Why do CNNs require fewer parameters than MLPs?

CNNs generally require fewer parameters than MLPs because convolutional layers use local connectivity and parameter sharing. A single filter is reused across different spatial locations of an image instead of assigning separate weights to every input pixel and neuron connection. This makes CNNs more parameter-efficient for image processing.

13. References

1. Goodfellow, I., Bengio, Y., and Courville, A., *Deep Learning*, MIT Press.
2. Bishop, C. M., *Pattern Recognition and Machine Learning*, Springer.
3. Haykin, S., *Neural Networks and Learning Machines*, Pearson.

4. CIFAR-10 Dataset.
5. TensorFlow/Keras Documentation.